Maser Engineering — Documentation technique

Application de gestion de projets d'ingénierie · mock-up v1.0 · 18 octobre 2026 · référence pour le développement

1. Objet et périmètre

Ce document décrit l'application « Maser Engineering — Project Management » telle que livrée dans le mock-up (`maser-engineering-app.html`). Il sert de référence pour développer la version de production : fonctionnalités, matrice des droits, modèle de données, règles métier, routes, système de thème, et écarts entre le mock-up et le produit cible.

Le mock-up est un **front-end seul** : aucune API, aucun stockage serveur, données de démonstration en mémoire. Tout rechargement remet les données à zéro ; seules les préférences d'affichage (thème, palette, taille, vue par défaut) sont conservées dans le navigateur.

Légende utilisée dans ce document

- ✅ implémenté dans le mock-up : comportement à reproduire tel quel
- 🟡 simulé : l'interface existe mais n'a aucun effet réel
- ⬜ à construire : absent du mock-up

Conventions : interface en anglais avec quelques libellés français (Avenants, Réunions, Planning, Hiérarchie), devise EUR, dates au format britannique, dates stockées en `YYYY-MM-DD` .

2. Vue d'ensemble du produit

Outil de pilotage de projets pour un bureau d'études : suivi des projets, tâches, campagnes de tests, notes techniques, avenants contractuels, réunions, planning des équipes et espace client.

Quatre rôles : Admin, Manager, Collaborator, Client. S'y ajoutent deux notions transverses qui ne sont pas des rôles : le **chef de projet** (désigné par projet) et le **supérieur hiérarchique** (lien entre membres).

Modules

Module	Rôle fonctionnel
Login	Page de connexion (visuelle)
Dashboard	Synthèse : KPI, projets actifs, échéances, réunions, mes tâches, activité
Projects	Liste, création et détail d'un projet (8 onglets)
Tasks	Tâches transverses, vues Board et List
Tests	Campagnes et cas de test, résultats
Gantt Chart	Planning du portefeuille et des projets
Notes	Notes techniques, visites de site, décisions, messages client
Amendments	Avenants (périmètre, planning, budget) et circuit d'approbation
Meetings	Réunions à venir et passées
Team Schedule	Planning des collaborateurs par projet (Admin, Manager)
My Schedule	Planning personnel et agenda (Collaborator)
Organisation	Organigramme éditable (Admin)
Client Space	Vue client : avancement, jalons, réunions, messages, avenants
Settings	Profil, apparence (thème fonctionnel), notifications, outils par rôle

3. Architecture du mock-up

3.1 Technologies

Le code source d'origine est une application **Next.js (App Router)** avec **React**, **Tailwind CSS v4** et des composants de type **shadcn/ui** (primitives base-ui). Pour le livrable, le projet a été converti en fichier HTML autonome : bundle JavaScript unique (React 19 compilé avec esbuild), CSS Tailwind compilé, icônes Lucide, primitives d'interface réécrites sans dépendance externe (dialogues, menus, sélecteurs, onglets, infobulles) et routeur à base de hash (`#/chemin?query`).

Pour la version de production, il est recommandé de repartir du code source Next.js d'origine plutôt que du fichier HTML généré : le HTML est un livrable de démonstration, pas une base de maintenance.

3.2 Organisation du code

```
src/
  main.tsx                routes, garde d'accès par rôle (RoleGate), racine
  globals.css             tokens de couleur (palette Maser + 5 palettes),
animations
  lib/
    types.ts              modèle de données (types et énumérations)
    seed.ts               données de démonstration (membres, projets,
tâches…)
    store.tsx             état global (StoreProvider) + règles d'accès
exposées
    permissions.ts        moteur de droits (hiérarchie, lecture, écriture,
direction)
    settings.tsx          préférences d'affichage (thème, palette…) +
persistance
    format.ts, utils.ts   dates, devise, échéances, classes CSS
  components/
    shell/                barre latérale, barre supérieure, recherche,
coquille
    dashboard/, projects/ tableau de bord, liste et détail de projet
    modules/              tâches, tests, Gantt, notes, avenants, réunions,
                          planning d'équipe, organigramme
    dialogs/              création de projet, tâche, édition, avenant,
réunion…
    client/, settings/, auth/   espace client, paramètres, connexion
    shared/, ui/          badges, avatars, en-têtes ; primitives
d'interface
```

3.3 État et navigation

- **`StoreProvider`** (état en mémoire, `useState`) : membres, projets, tâches, tests, notes, avenants, réunions, jalons, utilisateur courant. Il expose les données **déjà filtrées** selon l'utilisateur et les fonctions de droits (section 5).
- **`SettingsProvider`** : thème, palette, taille d'interface, vue par défaut des tâches. Persistance dans `localStorage` (clé `maser-prefs-v1`), appliquée avant le premier rendu pour éviter tout flash.
- **Routage** : hash. Un hash vide ouvre la page de login. L'onglet d'un projet se passe en paramètre (`#/projects/p1?tab=tasks`).

- **Sélecteur de rôle** (menu du profil) : fonctionnalité de prototype uniquement ; permet de changer de rôle puis de personne pour un même rôle. À remplacer par l'authentification réelle.

3.4 Limites du mock-up à ne pas reproduire

Sujet	Constat dans le mock-up	Attendu en production
Date du jour	Constante figée `TODAY = 2026-10-16`	Date serveur réelle
Code projet	`MSR-25NN` calculé sur le nombre de projets	Séquence serveur unique, sans collision
Identifiants	Générés côté client	UUID générés par le serveur
Contrôle des droits	Surtout dans l'interface ; le store ne protège que l'écriture de tâches et de tests	Tous les droits vérifiés **côté serveur**
Notifications	Liste statique	Événements réels, en temps réel ou par email
Validation	Quasi inexistante	Validation de schéma côté serveur
Historique	Aucun	Journal d'audit
Avenant refusé	Statut inexistant	À définir (voir section 13)

4. Rôles, hiérarchie et notions de droits

Notion	Définition	Donnée
Admin	Accès total, administre l'organisation	`Member.role = Admin`
Manager	Pilote ses projets et son équipe	`Member.role = Manager`
Collaborator	Travaille sur les projets auxquels il est affecté	`Member.role = Collaborator`
Client	Externe, accès limité à l'Espace Client	`Member.role = Client`, `company`
Membre de projet	Personne affectée à un projet	`Project.memberIds`

Notion	Définition	Donnée
Manager du projet	Manager figurant dans les membres du projet	`role = Manager` et `∈ memberIds`
Chef de projet	Un collaborateur désigné sur un projet (un seul par projet)	`Project.leadId`
Supérieur hiérarchique	Responsable direct d'un membre	`Member.supervisorId`
Équipe	Soi-même plus tous les subordonnés, directs et indirects	dérivée
Périmètre	Projets où l'on est membre, ou membre via un subordonné	dérivé

Fonctions de droits (`permissions.ts`)

Fonction	Règle
`subordinatesOf(u)`	Tous les subordonnés de `u`, parcours transitif de `supervisorId`
`teamOf(u)`	Admin : tous les non-clients. Sinon `u` plus ses subordonnés
`hasProjectAccess(p, u)`	Admin, ou `u` ou l'un de ses subordonnés ∈ `p.memberIds`
`canViewProject(p, u)`	Admin et Manager : tous. Client : `p.client = u.company`. Collaborator : `hasProjectAccess`
`canEditProject(p, u)`	Admin : oui. Client : non. Sinon `hasProjectAccess`
`canLead(p, u)`	Admin, ou parmi `u` et ses subordonnés : un Manager ∈ `memberIds` **ou** le `leadId`
`canAssignLead(p, u)`	Admin, ou Manager avec `canLead` sur `p`
`canSeeNote(n, u)`	Collaborator : projet de la note accessible. Manager : il est membre du projet **ou** l'auteur est dans son équipe. Admin et Client : voir section 5

Principe d'héritage : un supérieur dispose *au moins* des droits de chacun de ses subordonnés sur leurs projets. Si un subordonné est chef de projet, son supérieur obtient aussi les droits de direction sur ce projet.

5. Matrice des droits

Légende des niveaux : **Tous** = tous les projets · **Écriture** = projets du périmètre d'écriture (membre ou supérieur d'un membre) · **Direction** = projets où l'utilisateur a `canLead` · **Siens** = projets visibles pour lui.

5.1 Matrice principale

Capacité	Admin	Manager	Chef de projet	Collaborator	Client
Voir les projets	Tous	Tous (lecture seule hors périmètre)	Siens	Siens	Ceux de son entreprise, via l'Espace Client
Créer un projet	Oui	Oui (devient membre)	Oui (devient membre)	Oui (devient membre)	Non
Créer et modifier une tâche (statut, édition, glisser-déposer)	Tous	Écriture	Écriture	Écriture	Non
Valider un test (Passed, Failed, Pending)	Tous	Écriture	Écriture	Écriture	Non
Lire les notes	Toutes	Projets dont il est membre + notes de son équipe	Notes de ses projets	Notes de ses projets	Messages de catégorie Client
Ajouter une note	Tous	Projets dont il est membre	Écriture	Écriture	Message (catégorie Client)
Commenter une note	Oui	Écriture	Écriture	Écriture	Oui
Modifier ou supprimer une note	Toutes	Les siennes	Les siennes	Les siennes	Non
Créer un avenant	Oui	Oui	Oui	Oui	Non
Soumettre un avenant (Draft vers Pending)	Oui	Oui	Oui	Oui	Non
Approuver un avenant (Pending vers Approved)	Tous	Direction	Son projet	Non	Non
Planifier une réunion	Tous	Direction	Son projet	Non	Non

Capacité	Admin	Manager	Chef de projet	Collaborator	Client
Consulter les réunions	Toutes	Toutes	Siens	Siens	Siens, lecture seule
Désigner un chef de projet	Tous	Direction (Manager)	Non	Non	Non
Planning d'équipe (Team Schedule)	Tous les collaborateurs	Son équipe	Non	Non	Non
Mon planning (My Schedule)	Non	Non	Oui	Oui	Non
Organisation (modifier les supérieurs)	Oui	Non	Non	Non	Non
Aperçu de l'Espace Client	Oui	Oui	Oui	Oui	Son espace
Recherche globale	Tous	Tous	Siens	Siens	Non
Paramètres : profil, apparence, langue, notifications, sécurité	Oui	Oui	Oui	Oui	Oui

Le chef de projet est un Collaborator avec `leadId` sur un projet donné : il cumule les droits d'un collaborateur et les deux droits de direction (approbation des avenants, planification des réunions) **sur ce seul projet**. Il ne peut pas désigner un autre chef de projet.

5.2 Droits par page

Route	Admin	Manager	Collaborator	Client
`/` Dashboard, `/projects`, `/tasks`, `/tests`, `/gantt`, `/notes`, `/amendments`, `/meetings`	Oui	Oui	Oui (données de ses projets)	Redirigé vers `/client`
`/projects/:id`	Oui	Oui (« View only » hors périmètre)	Seulement ses projets, sinon écran d'accès refusé	Redirigé
`/team`	Oui	Oui (son équipe)	Écran « réservé »	Redirigé

Route	Admin	Manager	Collaborator	Client
`/my-schedule`	Écran « réservé »	Écran « réservé »	Oui	Redirigé
`/organisation`	Oui	Écran « réservé »	Écran « réservé »	Redirigé
`/client`	Aperçu	Aperçu	Aperçu	Oui
`/settings`	Oui	Oui	Oui	Oui
`/login` (hash vide)	Public	Public	Public	Public

5.3 Effet de la hiérarchie sur le jeu de démonstration

Écriture = **E**, Direction = **D** (inclut l'écriture), Lecture seule = **V**, invisible = **—**.

Utilisateur	p1 MSR-2401	p2 MSR-2402	p3 MSR-2403	p4 MSR-2404	p5 MSR-2305	p6 MSR-2209
Karim Maser (Admin)	D	D	D	D	D	D
Sophie Laurent (Manager)	D	E	D	V	D	E
Nora Haddad (Manager)	E	D	V	D	V	D
Julien Moreau (Collab.)	E	E	—	E	—	—
Amina Benali (Collab.)	E	E	E	—	—	E
Lucas Petit (Collab.)	E	—	E	—	E	—

Lecture du tableau : Sophie supervise Amina et Lucas. Amina est membre de p2 et p6, donc Sophie obtient l'écriture sur ces projets sans en être manager. Elle reste en lecture seule sur p4, qui n'a aucun lien avec son équipe. Si Nora désigne Amina chef de projet de p2, Sophie obtient la direction (D) sur p2 ; elle la perd si Amina est retirée.

6. Règles de gestion

6.1 Projets

- **Statuts** : `In Progress`, `Closed`, `Archived`. **Priorités** : `High`, `Medium`, `Low`.

- **Création** (dialogue « Create project ») : nom, client, priorité, dates de début et de fin, tags, budget, description. Le créateur devient l'unique membre (`memberIds = [créateur]`). Réservé aux rôles non-client.
- **Code projet** : `MSR-` suivi d'un numéro. Dans le mock-up il est dérivé du nombre de projets existants ; en production, séquence serveur.
- **Progression** (0 à 100 %) : moyenne des tâches du projet, chaque tâche comptant `Done` = 1, `Review` = 0,75, `In Progress` = 0,4, `To Do` = 0. Un projet sans tâche vaut 0 %.
- **État d'échéance** : `Delivered` si le projet est terminé, `Overdue` si la date de fin est passée, `Due soon` à 14 jours ou moins, sinon `On track`. Les mêmes seuils servent aux échéances de tâches.
- **Projets en lecture seule** : un utilisateur qui voit un projet sans pouvoir l'écrire (Manager hors périmètre) voit le badge « View only » dans l'en-tête et des mentions « View only » sur les onglets Tasks, Tests et Notes.

6.2 Tâches

- **Statuts** : `To Do`, `In Progress`, `Review`, `Done`. Les transitions sont libres (n'importe quel statut vers n'importe quel autre, y compris la réouverture d'une tâche terminée).
- **Création** : réservée au périmètre d'écriture et aux projets `In Progress`. Le responsable proposé est choisi parmi les membres du projet.
- **Modification** : titre, statut, priorité, responsable, dates de début et d'échéance. Disponible dans le Board (menu « … » de la carte, glisser-déposer) et dans la List (menu sur le badge de statut, bouton crayon).
- **Filtres** : projet, priorité, « Assigned to me » (activé par défaut pour un Collaborator).
- **Droit d'édition** : toute personne du périmètre d'écriture peut modifier **toutes** les tâches du projet, pas seulement les siennes.

6.3 Tests

- **Statuts** : `Passed`, `Failed`, `Pending`. Les cas de test sont regroupés en **campagnes** (projet + nom de campagne), avec une barre de répartition par campagne.
- **Valider un test** : menu « Set result » sur le badge ; la date `lastRun` est mise à jour avec la date du jour.
- La **création** de cas de test n'existe pas dans le mock-up (⬜).

6.4 Notes

- **Catégories** : `Technical`, `Site Visit`, `Decision`, `Client`. Les notes de catégorie `Client` sont les messages échangés avec le client ; ce sont les seules visibles dans l'Espace Client.

- **Visibilité** : voir la matrice 5.1. Pour un Manager : notes des projets dont il est membre, plus celles écrites par les membres de son équipe. Un Collaborator ne voit que les notes de ses projets.
- **Édition et suppression** : l'auteur ou un Admin. **Commentaires** : réservés au périmètre d'écriture, ou à un client sur ses messages.

6.5 Avenants

- **Référence** : `AV-NNN`, séquentielle **par projet**.
- **Types** : `Scope`, `Schedule`, `Budget`. Champs chiffrés : `amount` (montant en euros) et `daysImpact` (impact en jours).
- **Circuit** : `Draft` puis `Pending Approval` (bouton « Submit », tout rôle non-client) puis `Approved` (bouton « Approve », réservé à la direction du projet). Aucun statut de refus ni retour arrière n'existe dans le mock-up.
- **Indicateurs** : somme des montants et des jours des avenants approuvés (« Approved budget impact », « Approved schedule impact »).
- **Client** : voit les avenants non `Draft` de ses projets, en lecture seule.

6.6 Réunions

- **Champs** : titre, date et heure, durée en minutes, lieu ou lien, participants. Dans le mock-up, les quatre premiers membres du projet sont ajoutés automatiquement comme participants ; en production, prévoir une sélection.
- **Création** : réservée à la direction du projet. Un utilisateur sans ce droit voit le message « Only managers and project leads can schedule meetings ». La liste des projets proposés dans le dialogue est limitée à ceux sur lesquels il a la direction.
- **Affichage** : « Upcoming » et « Past meetings » selon la date du jour. Le client voit les réunions de ses projets en lecture seule.

6.7 Planning d'équipe (Team Schedule)

Le planning est **calculé** à partir des tâches et des affectations, sans saisie dédiée.

- Pour chaque personne et chaque projet : la période va de la **première date de début à la dernière échéance** de ses tâches sur ce projet.
- Si la personne est membre du projet sans aucune tâche : la période reprend celle du projet, affichée hachurée (« Assigned, no task yet »).
- Les chevauchements sont empilés sur plusieurs lignes ; la ligne « Today » repère la date du jour.
- **Statut d'une affectation** : `Current`, `Upcoming` ou `Completed` selon la date du jour. Par personne : « N projects now » ou « Available ».

- **Périmètre** : Admin voit tous les non-clients ; Manager voit lui-même et ses subordonnés.
- **Filtres** : « Active projects » (par défaut), « All projects » ou un projet précis. Deux vues : Timeline et List.

6.8 Mon planning (My Schedule)

Chronologie personnelle (mêmes règles de calcul) plus un **agenda** regroupant par jour les tâches ouvertes (échéance) et les réunions à venir de la personne, avec marquage « Overdue » des retards.

6.9 Chef de projet

- Un seul par projet (`leadId`), choisi parmi les membres du projet de rôle Collaborator.
- Désignation et retrait depuis l'onglet Team du projet, par un Admin ou un Manager ayant la direction du projet.
- Effets : approbation des avenants et planification des réunions sur ce projet ; l'héritage hiérarchique s'applique à ses supérieurs.

6.10 Organisation

- Chaque personne a au plus un supérieur ; les personnes sans supérieur sont au sommet.
- Le sélecteur de supérieur exclut la personne elle-même et tous ses subordonnés, ce qui **empêche les boucles**.
- Seul l'Admin modifie la hiérarchie. Toute modification change immédiatement le planning d'équipe, les droits hérités et les notes visibles.

6.11 Espace Client

- Un client voit les projets où `Project.client` correspond à sa `company`, ou auxquels il est rattaché.
- Contenu : avancement, jalons, réunions (lecture seule), messages (notes `Client`), avenants non brouillons, contacts (Managers et Admin du projet).
- Les tâches, tests et notes internes ne sont jamais exposés au client.
- Les rôles internes disposent d'un **aperçu** de l'Espace Client.

6.12 Recherche globale et notifications

- **Recherche** : projets (nom, client, code, tags) et tâches (titre) dans le périmètre visible ; les résultats ouvrent le projet ou son onglet Tasks.
- **Notifications** 🟡 : cloche avec compteur et quatre événements statiques.

7. Modèle de données

Tous les identifiants sont des chaînes ; les dates sont au format `YYYY-MM-DD` (les dates-heures en ISO 8601).

7.1 Entités du mock-up

Entité	Champs	Remarques
Member	`id`, `name`, `initials`, `role` (Admin, Manager, Collaborator, Client), `title`, `company?`, `supervisorId?`	`company` renseigné pour les clients ; `supervisorId` crée la hiérarchie
Project	`id`, `code`, `name`, `client`, `description`, `status`, `priority`, `startDate`, `endDate`, `tags[]`, `memberIds[]`, `leadId?`, `budget`	`leadId` doit appartenir à `memberIds`
Task	`id`, `projectId`, `title`, `status`, `priority`, `assigneeId`, `startDate`, `dueDate`	`dueDate` supérieure ou égale à `startDate`
TestCase	`id`, `projectId`, `campaign`, `name`, `status`, `priority`, `assigneeId`, `lastRun`	Campagne = libellé libre, regroupé par projet
Note	`id`, `projectId`, `authorId`, `category`, `content`, `createdAt`, `comments[]`	Catégorie `Client` = message client
NoteComment	`id`, `authorId`, `content`, `createdAt`	Imbriqué dans la note
Amendment	`id`, `projectId`, `ref`, `title`, `type`, `status`, `amount`, `daysImpact`, `requestedBy`, `createdAt`	`ref` unique par projet
Meeting	`id`, `projectId`, `title`, `start`, `durationMin`, `location`, `attendeeIds[]`	
Milestone	`id`, `projectId`, `title`, `date`	Affichés dans le Gantt et l'Espace Client

7.2 Relations

- Un projet a plusieurs membres (relation plusieurs-à-plusieurs `Project.memberIds`), un chef de projet optionnel, et plusieurs tâches, tests, notes, avenants, réunions et jalons.
- Un membre a au plus un supérieur ; la hiérarchie est un arbre (forêt) sans cycle.
- Une tâche ou un test a un responsable qui devrait être membre du projet.
- Une réunion a plusieurs participants.

7.3 Données dérivées (non stockées)

Progression d'un projet, état d'échéance, jours restants, équipe d'un membre, périmètre de lecture et d'écriture, affectations du planning (section 6.7), KPI du dashboard.

7.4 Entités à créer pour la production ⬜

Entité	Usage
Utilisateur et identifiants	Authentification (email, mot de passe haché, double authentification) ; lien avec `Member`
Session	Liste des appareils connectés, déconnexion à distance
Journal d'audit	Qui a changé quoi (statut, droits, hiérarchie) ; alimente l'« Activity log »
Notification et préférences	Événements, canal (application, email), résumés, heures calmes
Absence	Congés et absences, à intégrer au planning
Délégation	Délégation temporaire d'approbation (dates, délégataire)
Intégration	Connexion Meet, Teams, calendriers, Slack
Paramètres d'espace de travail	Préfixe de code, devise, priorité par défaut, seuil d'approbation d'avenant
Pièce jointe	Documents liés aux projets, notes et avenants (absents du mock-up)

8. Pages et routes

Route	Écran	Paramètres	Accès
(hash vide), `/login`	Connexion	—	Public
`/`	Dashboard	—	Rôles internes
`/projects`	Liste des projets	—	Rôles internes
`/projects/:id`	Détail du projet	`tab` : overview, tasks, tests, gantt, notes, amendments, meetings, team	Selon périmètre
`/tasks`	Tâches (Board, List)	—	Rôles internes
`/tests`	Tests par campagne	—	Rôles internes
`/gantt`	Gantt du portefeuille	—	Rôles internes
`/notes`	Notes	—	Rôles internes
`/amendments`	Avenants	—	Rôles internes
`/meetings`	Réunions	—	Rôles internes
`/team`	Planning d'équipe	—	Admin, Manager
`/my-schedule`	Mon planning	—	Collaborator
`/organisation`	Organigramme	—	Admin
`/client`	Espace Client	—	Client, aperçu des autres
`/settings`	Paramètres	`section` : profile, appearance, language, notifications, security, team, availability, contacts, integrations, workspace, permissions, activity	Tous (sections selon rôle)

En production, remplacer le routage par hash par un routage d'URL classique et protéger chaque route côté serveur.

9. Fonctionnalités par module

9.1 Connexion ✅ (visuel) · 🟡 (logique)

- Page à deux panneaux (marque et formulaire), responsive : le panneau de marque disparaît sur mobile.
- Champs email et mot de passe (bouton pour afficher), case « Keep me signed in », lien « Forgot password? » 🟡, bouton « Continue with SSO » 🟡.
- « Sign in » ouvre l'application sans aucune vérification 🟡. « Sign out » (menu du profil) ramène à la connexion. Un hash vide ouvre cette page.

9.2 Navigation générale ✅

- **Barre latérale** : groupe Workspace (Dashboard, Projects, Tasks, Tests, Gantt Chart, Notes, Amendments, Meetings), plus selon le rôle Team Schedule (Admin, Manager), Organisation (Admin), My Schedule (Collaborator) ; groupe External (Client Space) ; bouton « New Project ». Le client ne voit que Client Space.
- **Mobile** : barre latérale en tiroir (bouton menu).
- **Barre supérieure** : recherche globale, cloche de notifications 🟡 (compteur et liste statique), menu du profil (changement de rôle et de personne : prototype uniquement ; Settings ; Sign out).
- L'encart « Prototype mode » de la barre latérale est à supprimer en production.

9.3 Dashboard ✅

- Salutation et date, lien « View timeline » vers le Gantt.
- **Quatre tuiles KPI** : projets actifs, échéances du mois, tâches en cours, tests en échec (avec sous-texte : clos ou archivés, en revue, en attente d'exécution).
- **Widgets** : projets actifs (cartes avec code, client, priorité, progression, tags, équipe, échéance, état), prochaines échéances, prochaines réunions, mes tâches ouvertes, activité récente (dernières notes).
- Toutes les données respectent le périmètre de lecture de l'utilisateur.

9.4 Projets ✅

- **Liste** : filtre texte (nom, client, tag), filtres de statut et de priorité, bascule grille ou liste, accès au détail.
- **Création** : dialogue (section 6.1).
- **Détail** : en-tête (code, nom, badges de statut, priorité, état d'échéance, tags, « View only »), carte de progression globale avec jours restants, puis huit onglets.

  - **Overview** : périmètre (client, dates, budget avec avenants), tuiles (tâches terminées sur total, en revue, tests réussis, tests en échec), Gantt du projet, carte Team.
  - **Tasks, Tests, Gantt, Notes, Amendments, Meetings** : les modules correspondants, filtrés sur le projet.
  - **Team** : membres avec rôle, « Reports to », badge « Project lead » et bouton « Make project lead » ou « Remove as project lead » selon les droits.
- Un collaborateur qui ouvre un projet hors périmètre voit « You don't have access to this project ».

9.5 Tâches ✅

- **Board** : quatre colonnes (To Do, In Progress, Review, Done) avec compteurs, bouton « + » par colonne, cartes déplaçables (titre, projet, priorité, échéance colorée selon l'état, avatar du responsable). Zone vide : « Drop tasks here ».
- **List** : tableau Task, Project, Status (menu), Priority, Assignee, Due, Actions (crayon).
- Dialogues « Add task » et « Edit task ». Mode « View only » sans aucune action d'écriture.
- La vue par défaut (Board ou List) se règle dans Settings.

9.6 Tests ✅

- Trois tuiles Passed, Failed et Pending (nombre et part des tests) qui servent aussi de filtres, une carte par campagne avec barre de réussite, et le tableau des cas avec statut modifiable (« Set result »), date de dernier passage, responsable, priorité.
- Compteur « campagnes · cas de test » et mention « View only » si aucun droit d'écriture.

9.7 Gantt ✅

- Vue portefeuille (un projet par ligne) ou vue projet (tâches par ligne, jalons sur la première ligne).
- Échelle mensuelle, ligne « Today », barres colorées selon le statut avec pourcentage, jalons en losange, colonne de libellés fixe, défilement horizontal.

9.8 Notes ✅

- Composeur avec catégorie ; filtres par catégorie avec compteurs ; liste (auteur, catégorie, date, lien vers le projet) ; commentaires ; modification et suppression (auteur ou Admin).
- En vue client : composeur « Leave a message for the project team », uniquement la catégorie `Client`.
- Sans droit d'écriture : message « View only » à la place du composeur.

9.9 Avenants ✅

- Quatre tuiles : impact budget et impact délai des avenants approuvés, avenants en attente d'approbation, brouillons.
- Tableau : référence, titre, projet, type, budget, délai, demandeur, statut, action (Submit ou Approve selon l'état et les droits).
- Dialogue « New amendment » (titre, projet, type de révision, impact budget en euros, impact délai en jours).

9.10 Réunions ✅

- Sections Upcoming et Past meetings ; cartes avec pastille de date, heure, durée, lieu, participants.
- Dialogue « Schedule meeting » (section 6.6) ; message explicatif pour les rôles sans droit.

9.11 Planning d'équipe, Mon planning, Organisation ✅

- **Team Schedule** (Admin, Manager) : chronologie par collaborateur et liste, filtre de projets, légende des couleurs par projet, infobulles (projet, période, nombre de tâches), liens vers les projets. Règles en 6.7.
- **My Schedule** (Collaborator) : chronologie personnelle et agenda (6.8).
- **Organisation** (Admin) : arbre hiérarchique avec, pour chaque personne, rôle, effectif en dessous, nombre de projets et sélecteur de supérieur (6.10).

9.12 Espace Client ✅

Onglets par projet du client ; carte d'avancement ; jalons (cochés selon la date) ; réunions à venir ; messages avec composeur ; contacts ; avenants non brouillons avec impact.

9.13 Paramètres

Accès par le menu du profil. Navigation latérale (défilement horizontal sur mobile) ; la section ouverte se règle par `?section=`.

Section	Rôles	Contenu	État
Profile	Tous	Identité, signature, rôle et supérieur en lecture seule	🟡 non enregistré

Section	Rôles	Contenu	État
Appearance	Tous	Mode clair, sombre ou système ; 6 palettes ; taille d'interface ; vue par défaut des tâches ; réinitialisation	✅ fonctionnel, mémorisé
Language & region	Tous	Langue, formats de date et d'heure, semaine, fuseau	🟡
Notifications	Tous	Matrice événements × canaux, résumé email, heures calmes ; liste d'événements propre au client	🟡
Security	Tous	Mot de passe, double authentification, sessions actives	🟡
Team preferences	Manager	Rappels d'échéance, résumé hebdomadaire, alerte de surcharge, délégation d'approbation	🟡 (la délégation ne change aucun droit)
Availability	Collaborator, Manager	Jours et horaires de travail, congés et absences	🟡 (non lié au planning)
Your contacts	Client	Responsables Maser de ses projets	✅ données réelles
Integrations	Admin, Manager	Meet, Teams, Google Calendar, Outlook, Slack	🟡
Workspace	Admin	Valeurs par défaut, seuil d'approbation, lien Organisation, export JSON	🟡 sauf export ✅
Roles & permissions	Admin	Tableau des droits par rôle	✅ statique (à tenir synchronisé avec le code)
Activity log	Admin	Journal filtrable par type	🟡 événements d'exemple

10. Thème et design system

10.1 Mécanisme

Toutes les couleurs passent par des variables CSS (tokens) définies sur `:root`. Les valeurs claires et sombres sont déclarées ensemble avec `light-dark()` et `color-scheme: light dark` : l'interface suit le thème du système, ou celui choisi dans Settings.

Attribut sur `<html>`	Effet	Valeurs
`data-theme`	Force le mode ; absent = suit le système	`light`, `dark`
`data-palette`	Palette de couleurs ; absent = palette Maser	`ocean`, `forest`, `sunset`, `violet`, `graphite`
`data-density`	Taille de l'interface (la base `font-size` du document, tout est en rem)	`compact` (14,5 px), `large` (17 px) ; absent = 16 px

Les préférences sont enregistrées dans `localStorage` (`maser-prefs-v1`) et appliquées avant le premier rendu.

10.2 Tokens de la palette Maser (référence : fichier Mock-ups)

Token	Clair	Sombre	Usage
`--background`	`#f6f6f6`	`#1c1b1b`	Fond de page
`--foreground`	`#2b2929`	`#f6f6f6`	Texte
`--card`, `--popover`	`#ffffff`	`#272525`	Cartes, menus
`--primary`	`#e63641`	`#e63641`	Action principale, onglet actif, progression
`--alt`	`#4f4c4d`	`#b5b2b3`	Teinte neutre des badges « In Progress »
`--secondary`, `--muted`, `--accent`	`#f6f6f6`	`#333031`	Fonds secondaires, survols
`--muted-foreground`	`#6f6c6d`	`#aaa7a8`	Texte secondaire
`--border`, `--input`	`#e6e5e5`	`#3d3a3b`	Bordures

Token	Clair	Sombre	Usage
`--success`	`oklch(0.6 0.13 163)`	idem	Réussi, approuvé, dans les temps
`--warning`	`oklch(0.75 0.15 70)`	idem	Moyen, bientôt échu, en attente
`--destructive`	`oklch(0.58 0.22 27)`	idem	Haut, en retard, échec
`--chart-5`	`oklch(0.6 0.1 240)`	idem	Teinte bleue (avatars, messages client)
`--sidebar`	`#4f4c4d`	`#4f4c4d`	Barre latérale (identique en clair et sombre)
`--sidebar-accent`	`#625f60`	`#625f60`	Élément actif ou survolé de la barre latérale

Les teintes de badges dérivent de ces tokens : fond à 12 % de la teinte, bordure à 25 %, texte mélangé à 65 % avec la couleur de texte (`--*-ink`). Rayon de base : 0,5 rem.

10.3 Palettes alternatives

Chaque palette redéfinit les neutres, la couleur principale et la barre latérale ; les couleurs de statut (succès, alerte, erreur) restent communes.

Palette	Couleur principale	Ambiance
Maser (défaut)	`#e63641`	Gris chauds et rouge signal
Ocean	`#2f6fe4`	Ardoise froide, barre latérale `#1e293b`
Forest	`#1f8a5b`	Tons pierre, barre latérale `#24332b`
Sunset	`#d9480f`	Sable chaud, barre latérale `#3b322b`
Violet	`#7c3aed`	Gris lilas, barre latérale `#2e2a40`
Graphite	`#2b2929` (clair), `#f6f6f6` (sombre)	Monochrome sans accent

Pour ajouter une palette : déclarer un bloc `:root[data-palette=nom]` qui redéfinit les tokens, puis l'ajouter à la liste `PALETTES` de `lib/settings.tsx` (nom, description, aperçu clair et sombre).

10.4 Codes couleur sémantiques

Teinte	Token	Exemples
Gris neutre (alt)	`--alt`	Projet ou tâche « In Progress » (badge), catégorie Technical
Vert	`--success`	On track, Approved, Passed, Done, catégorie Site Visit
Ambre	`--warning`	Medium, Due soon, Pending, catégorie Decision
Rouge	`--destructive`	High, Overdue, Failed
Gris moyen	`--muted-foreground`	Low, Draft, Closed, Delivered
Rouge de marque	`--primary`	Barres de progression, tâches en cours dans le Gantt, onglet actif
Bleu	`--chart-5`	Catégorie Client, messages client

Avatars : couleur attribuée selon l'identifiant parmi primaire, vert, bleu, ambre foncé `oklch(0.62 0.14 70)` et gris `#4f4c4d`, texte blanc.

10.5 Composants

- **Primitives** (`components/ui`) : Button (variantes default, outline, secondary, ghost, destructive, link ; tailles xs à lg et icônes), Badge, Card, Input, Textarea, Label, Select, Dropdown menu, Dialog, Sheet, Tabs, Table, Tooltip, Avatar, Progress, Switch, Separator.
- **Partagés** : en-tête de page (PageHeader, ModulePage), état vide, bascule de vue, barre de progression, badges de statut, priorité et échéance, avatar de membre.
- **Accessibilité** : rôles ARIA (dialog, menu, listbox, tablist, switch, radiogroup, progressbar), fermeture par Échap, navigation aux flèches dans les menus, listes et onglets, piégeage du focus dans les dialogues, libellés accessibles sur les boutons d'icône.
- **Responsive** : points de rupture Tailwind (sm 640, md 768, lg 1024, xl 1280). Barre latérale fixe à partir de lg, tiroir en dessous ; tableaux et Gantt en défilement horizontal.
- Dans le livrable HTML, ces primitives sont réécrites sans base-ui ; en production, reprendre les composants shadcn/base-ui du code Next.js d'origine.

11. Réel et simulé : état du mock-up

Domaine	État	Remarque
Authentification	🟡	Aucune vérification, « Sign in » ouvre l'application
Persistance des données	🟡	Mémoire du navigateur, perdue au rechargement
Droits de lecture et d'écriture	✅ interface, ⬜ serveur	Moteur dans `permissions.ts` ; l'écriture de tâches et de tests est aussi protégée dans le store, pas les avenants, réunions, chef de projet ni hiérarchie
Hiérarchie et héritage	✅	Calculés à la volée à chaque rendu
Thème, palettes, taille, vue par défaut	✅	Seules préférences réellement mémorisées
Recherche globale	✅	Filtre côté client sur projets et tâches
Export des données	✅	JSON généré côté client depuis la mémoire
Notifications	🟡	Liste statique, aucun envoi
Intégrations, langue, profil, sécurité	🟡	Écrans sans effet
Planning et agenda	✅	Calculés à partir des tâches ; les absences ne sont pas prises en compte
Journal d'activité	🟡	Événements d'exemple
Pièces jointes, temps réel, e-mails	⬜	Hors périmètre du mock-up

12. Architecture cible et API proposée

Cette section est une **recommandation** pour la version de production ; le mock-up n'en implémente aucune partie serveur.

12.1 Principes

1. **Les droits vivent côté serveur.** Reprendre `permissions.ts` comme module de politiques (une fonction par capacité, avec tests unitaires) et l'appliquer à chaque requête. L'interface ne fait que masquer ce que le serveur refuserait.
2. **Filtrer à la requête, pas à l'affichage.** Les listes (projets, tâches, tests, avenants, réunions, notes) sont filtrées en base selon le périmètre de l'utilisateur ; un collaborateur ne reçoit jamais les projets qui ne sont pas les siens.
3. **Hiérarchie** : `Member.supervisorId` en base, avec parcours récursif (`WITH RECURSIVE`) ou table de fermeture pour obtenir les subordonnés ; mettre le résultat en cache et l'invalider à chaque changement de hiérarchie.
4. **Un point unique de décision** par capacité : `canView` , `canEdit` , `canLead` , `canAssignLead` , `canSeeNote` , `canSeeSchedule` . Les mêmes fonctions servent l'API, les exports et les notifications.
5. **Intégrité** : `leadId` doit être un membre du projet ; pas de cycle dans la hiérarchie ; `dueDate` supérieure ou égale à `startDate` ; référence d'avenant unique par projet.
6. **Audit** : toute écriture sensible (statut, approbation, chef de projet, hiérarchie) écrit une ligne d'audit, qui alimente le journal d'activité.
7. **Dates** : stocker les dates simples en `date` , les instants en UTC ; la date du jour vient du serveur.

12.2 Pile suggérée

Base PostgreSQL ; API REST (ou tRPC) ; authentification OIDC avec SSO d'entreprise et double authentification ; sessions révocables (liste des appareils) ; file de messages pour notifications et e-mails ; stockage objet pour les pièces jointes ; front-end Next.js d'origine.

12.3 Endpoints proposés

Méthode et chemin	Rôle	Autorisation
`POST /auth/login` , `POST /auth/logout`	Connexion, déconnexion	Public, puis session
`GET /me`	Profil, rôle, équipe, périmètre	Authentifié
`GET /me/sessions` , `DELETE /me/sessions/:id`	Appareils connectés	Soi-même

Méthode et chemin	Rôle	Autorisation
`PATCH /me/preferences`, `/me/password`, `/me/2fa`	Préférences et sécurité	Soi-même
`GET /members`	Annuaire	Authentifié (restreindre selon les besoins de confidentialité)
`PATCH /members/:id/supervisor`	Changer le supérieur	Admin ; refuse un cycle
`GET /projects`	Projets visibles	`canView` appliqué en requête
`POST /projects`	Créer un projet	Rôles non-client
`GET /projects/:id`	Détail	`canView` sinon 403
`PATCH /projects/:id/lead`	Désigner ou retirer le chef de projet	`canAssignLead`
`GET /tasks`, `GET /projects/:id/tasks`	Lister	`canView`
`POST /projects/:id/tasks`	Créer une tâche	`canEdit`, projet `In Progress`
`PATCH /tasks/:id`	Modifier (statut, champs)	`canEdit` du projet
`PATCH /tests/:id/result`	Valider un test (met à jour `lastRun`)	`canEdit`
`POST /projects/:id/tests`	Créer un cas de test ⬜	`canEdit`
`GET /notes`, `POST /projects/:id/notes`	Lire, ajouter	`canSeeNote` ; ajout selon 5.1
`PATCH` ou `DELETE /notes/:id`	Modifier, supprimer	Auteur ou Admin
`POST /notes/:id/comments`	Commenter	`canEdit`, ou client sur ses messages
`GET /amendments`, `POST /projects/:id/amendments`	Lister, créer	`canView`, puis tout rôle non-client
`POST /amendments/:id/submit`	Draft vers Pending	Rôle non-client
`POST /amendments/:id/approve`	Pending vers Approved	`canLead`

Méthode et chemin	Rôle	Autorisation
`GET /meetings`, `POST /projects/:id/meetings`	Lister, planifier	`canView` ; création `canLead`
`GET /schedule/team`	Planning d'équipe	Admin (tous), Manager (son équipe)
`GET /schedule/me`	Mon planning et agenda	Collaborator
`GET /client/projects` et sous-ressources	Espace Client	Client de l'entreprise concernée
`GET /search?q=`	Recherche	Périmètre de l'utilisateur
`GET` ou `PATCH /workspace/settings`	Valeurs par défaut	Admin
`GET /audit-log`	Journal d'activité	Admin
`GET /export`	Export des données	Admin

12.4 Conventions transverses

- **Erreurs** : 401 non authentifié, 403 droit manquant (le mock-up affiche « You don't have access » plutôt que « not found »), 404 ressource inexistante, 409 conflit (cycle, référence existante), 422 validation.
- **Concurrence** : numéro de version (ETag) sur tâches, tests et avenants pour éviter les écrasements silencieux.
- **Pagination et tri** sur toutes les listes ; filtres alignés sur ceux de l'interface (projet, priorité, statut, « assigné à moi »).
- **Notifications** : événements métier (tâche assignée, échéance proche, test en échec, avenant en attente, réunion, commentaire) publiés sur un bus, distribués selon les préférences de chaque utilisateur.
- **Planning** : conserver le calcul dérivé des tâches (6.7) ; l'enrichir avec les absences et les délégations quand elles existeront.

13. Points ouverts et décisions à valider

Chaque ligne indique le comportement actuel du mock-up et ce que nous recommandons pour la production. Ces arbitrages ont été faits pendant la conception du mock-up et méritent une validation explicite.

#	Sujet	Comportement actuel	Recommandation
1	L'Admin peut-il planifier des réunions et approuver des avenants ?	Oui, sur tous les projets	Conserver : l'Admin a tous les droits
2	Un supérieur sans lien direct avec un projet	Il hérite des droits de ses subordonnés (écriture, voire direction si l'un est chef de projet)	Valider la règle ; sinon passer en lecture seule stricte
3	Tâches terminées	Modifiables, avec réouverture possible	Envisager un verrouillage après clôture du projet
4	Qui peut modifier une tâche	Tout membre du projet, quelle que soit l'affectation	Valider ; alternative : seulement ses tâches, plus la direction
5	Avenants	Pas de refus, d'annulation ni de seuil d'approbation appliqué	Ajouter le statut `Rejected` et le seuil défini dans Settings
6	Création de projet par un collaborateur	Autorisée ; il devient seul membre, donc personne d'autre que l'Admin ne peut approuver	Réserver la création aux Managers et à l'Admin, ou imposer un manager
7	Notes d'un manager	Projets dont il est membre, plus notes de son équipe	Valider cette formulation de « notes de ses équipes »
8	Chef de projet	Un seul par projet, uniquement un collaborateur	Valider ; prévoir éventuellement plusieurs chefs ou un manager délégué
9	Hiérarchie	Un seul supérieur par personne	Valider ; une matrice multi-supérieurs complexifierait les droits
10	Annuaire	Tous les rôles internes voient tous les membres	Restreindre si la confidentialité l'exige
11	Client	Rattaché par le texte `company`	Créer une entité Entreprise cliente avec plusieurs comptes

#	Sujet	Comportement actuel	Recommandation
12	Participants d'une réunion	Quatre premiers membres du projet, sans sélection	Ajouter une sélection des participants et des invités externes
13	Planning	Calculé à partir des tâches uniquement	Intégrer absences, jours fériés et charge (heures)
14	Lecture seule d'un manager	Voit tous les projets, budget compris	Valider la visibilité du budget
15	Fonctions absentes	Création de cas de test, pièces jointes, commentaires de tâche, dépendances, suivi du temps	À prioriser dans le backlog
16	Langue	Interface en anglais avec libellés français	Prévoir l'internationalisation (FR et EN)

14. Plan de développement suggéré

Phase	Objectif	Contenu	Critère de fin
0. Fondations	Socle technique	Dépôt à partir du code Next.js, schéma de base, authentification et sessions, intégration continue	Connexion réelle et déploiement de recette
1. Données et droits	Lecture et écriture sécurisées	Entités du chapitre 7, module de politiques serveur, listes filtrées par périmètre, tâches et tests persistés	Tous les scénarios de l'annexe B passent contre l'API
2. Collaboration	Circuits métier	Notes et commentaires, avenants avec approbation, réunions, chef de projet	Parcours avenant et réunion complets avec droits serveur

Phase	Objectif	Contenu	Critère de fin
3. Pilotage	Équipes et organisation	Organigramme éditable, planning d'équipe et personnel, tableau de bord, Gantt, Espace Client	Plannings identiques au mock-up sur le jeu de démo
4. Paramètres et confort	Préférences réelles	Profil, notifications, sécurité, absences, délégation, préférences enregistrées côté serveur, export	Préférences conservées entre appareils
5. Ouverture	Intégrations et audit	Journal d'audit, e-mails, calendriers, Meet, Teams, Slack	Notifications et audit alimentés par de vrais événements
6. Qualité	Mise en production	Tests de bout en bout, accessibilité, performances, sauvegardes, supervision	Recette client signée

Annexe A. Jeu de données de démonstration

Date du jour figée dans le mock-up : 2026-10-16. Volumes : 7 membres, 6 projets, 19 tâches, 10 cas de test, 6 notes, 6 avenants, 6 réunions, 6 jalons.

Membres

Id	Nom	Rôle	Titre	Supérieur
u1	Karim Maser	Admin	Managing Director	—
u2	Sophie Laurent	Manager	Project Manager	u1
u3	Julien Moreau	Collaborator	Structural Engineer	u6
u4	Amina Benali	Collaborator	QA Engineer	u2
u5	Lucas Petit	Collaborator	Electrical Engineer	u2
u6	Nora Haddad	Manager	Project Manager	u1
u7	Marc Fontaine	Client (Helios Energy)	Operations Director	—

Projets

Id	Code	Nom	Client	Statut	Priorité	Membres
p1	MSR-2401	Solar Farm Substation Upgrade	Helios Energy	In Progress	High	u2, u3, u4, u5
p2	MSR-2402	Rail Depot Structural Retrofit	Nordline Rail	In Progress	Medium	u6, u3, u4
p3	MSR-2403	Water Treatment SCADA Migration	Aquaterra Water	In Progress	High	u2, u5, u4
p4	MSR-2404	Logistics Hub HVAC Design	Vantage Logistics	In Progress	Low	u6, u3
p5	MSR-2305	Battery Storage Feasibility Study	Helios Energy	Closed	Medium	u2, u5
p6	MSR-2209	Urban Footbridge Inspection	Cité Urbaine	Archived	Low	u6, u4

Profil par défaut à l'ouverture après connexion : Sophie Laurent (Manager). Profils de bascule du prototype : Karim (Admin), Julien (Collaborator), Marc (Client).

Annexe B. Scénarios de recette des droits

Ces scénarios ont été vérifiés par tests automatisés sur le mock-up ; ils servent de base de recette pour l'API et l'interface de production.

#	Scénario	Résultat attendu
1	Sophie ouvre Amendments	« Approve » sur l'avenant en attente de p1 ; aucun sur celui de p2
2	Nora ouvre Amendments	« Approve » sur l'avenant de p2 ; aucun sur celui de p1
3	Julien ouvre Amendments	Aucun « Approve » ; « New amendment » disponible
4	Karim ouvre Amendments	« Approve » sur les deux avenants en attente
5	Julien ouvre Meetings	Pas de « Schedule meeting » ; message explicatif
6	Sophie ouvre les réunions de p1, puis de p2	Création possible sur p1, absente sur p2
7	Nora désigne Amina chef de projet de p2	Badge « Project lead » ; Amina approuve et planifie sur p2 mais pas sur p1 ; Sophie hérite des droits sur p2 ; au retrait, Sophie les perd

#	Scénario	Résultat attendu
8	Un chef de projet ouvre l'onglet Team	Aucun bouton « Make project lead »
9	Julien ouvre Projects	Seuls p1, p2 et p4 ; p3 par adresse directe donne l'accès refusé ; absents du tableau de bord et de la recherche ; le dialogue de tâche ne propose que ses projets
10	Julien modifie des tâches de p1	Statut par menu, glisser-déposer, édition du titre, statut en vue List, édition en List ; modification d'une tâche d'un collègue après désactivation de « Assigned to me »
11	Julien valide un test de p1	Le résultat passe à Failed, `lastRun` mis à jour
12	Nora ouvre p3 (hors périmètre)	« View only » ; aucun ajout, menu de statut, glisser-déposer, validation de test ni composeur de notes
13	Sophie ouvre p4, p1 et p2	Lecture seule sur p4 ; modification sur p1 ; modification sur p2 par héritage via Amina
14	Team Schedule	Sophie voit 3 personnes, Nora 2, Karim 6 ; un collaborateur est refusé
15	Notes	Nora voit moins de notes que Karim ; un collaborateur voit seulement celles de ses projets
16	Karim rattache Julien à Sophie	Sophie voit 4 personnes dans son planning ; un manager est refusé sur Organisation
17	Marc ouvre Client Space	Projets p1 et p5 ; ni tâches, ni tests, ni notes internes ; avenants non brouillons
18	Thème	Dark : fond `#1c1b1b` ; palette Ocean : `--primary` = `#2f6fe4` ; conservation après rechargement ; Reset
19	Connexion	Hash vide : page de login ; « Sign in » : tableau de bord ; « Sign out » : login

Annexe C. Glossaire

Terme	Signification
Avenant (Amendment)	Modification contractuelle du périmètre, du planning ou du budget d'un projet
Réunion (Meeting)	Réunion de projet, visite de site ou revue client
Chef de projet (Project lead)	Collaborateur désigné sur un projet, avec droits d'approbation et de planification de réunions
Périmètre d'écriture	Projets où l'on est membre ou supérieur d'un membre
Direction	Droits d'approbation des avenants et de planification des réunions sur un projet
Planning	Vue des affectations des personnes dans le temps
Mock-up	Prototype front-end sans serveur décrit dans ce document